

Recursão e Recorrência

Definindo Recursão e Recorrência

ICMC USP SÃO CARLOS

Sumário

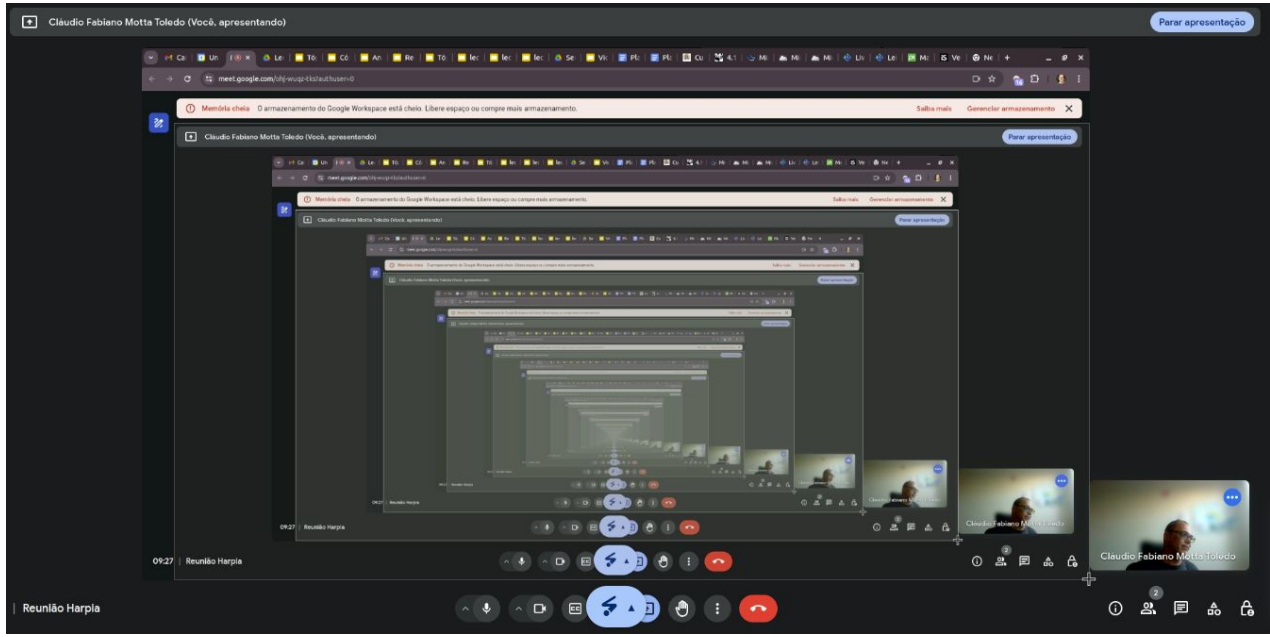
- Definindo Recursão e Recorrência
- Algoritmo Recursivo

Referências

- SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al. Análise de Algoritmos. Porto Alegre: SAGAH, 2021. E-book. p.25. ISBN 9786556901862. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9786556901862/.
- GERSTING, Judith L. Fundamentos Matemáticos para a Ciência da Computação. 7th ed. Rio de Janeiro: LTC, 2016. E-book. p.143. ISBN 9788521633303. Disponível em: https://app.minhabiblioteca.com.br/reader/books/9788521633303/.

Definindo Recursão e Recorrência

- Recursão é o processo de definir um objeto em função dele próprio.
- Esse objetos podem ser sequências, conjuntos, funções, algoritmos.



Definindo Recursão e Recorrência

Exemplo: Seja a sequência de potência de dois definida como $a_n=2^n$.

Recursivamente temos: $a_0=1$ e $a_n=2a_{n-1}$

$a_0=1$

$a_1=2a_0=2(1)=2$

$a_2=2a_1=2(2)=4$

$a_3=2a_2=2(4)=8$

$a_4=2a_3=2(8)=16$

$a_5=2a_4=2(16)=32$

...

$a_n=2a_{n-1}=2^n$

Definindo Recursão e Recorrência

- O item sendo definido aparece como parte da definição.
- As chamadas definição recorrente, definição por recorrência ou definição por indução apresentam duas partes:
  - **Passo base**: casos simples do item a ser definido são explicitamente estabelecidos.
  - **Passo Indutivo ou recorrente**: novos casos do item a ser definido são dados em função de casos anteriores.

Definindo Recursão e Recorrência

Exemplos:

Fatorial	Fibonacci	Potência 2
F(1)=1	F(0)=1	F(0)=1
F(n)=nF(n-1)	F(1)=1	F(1)=2
	F(n)=F(n-1)+F(n-2)	F(n)=2F(n-1)



Algoritmo Recursivo

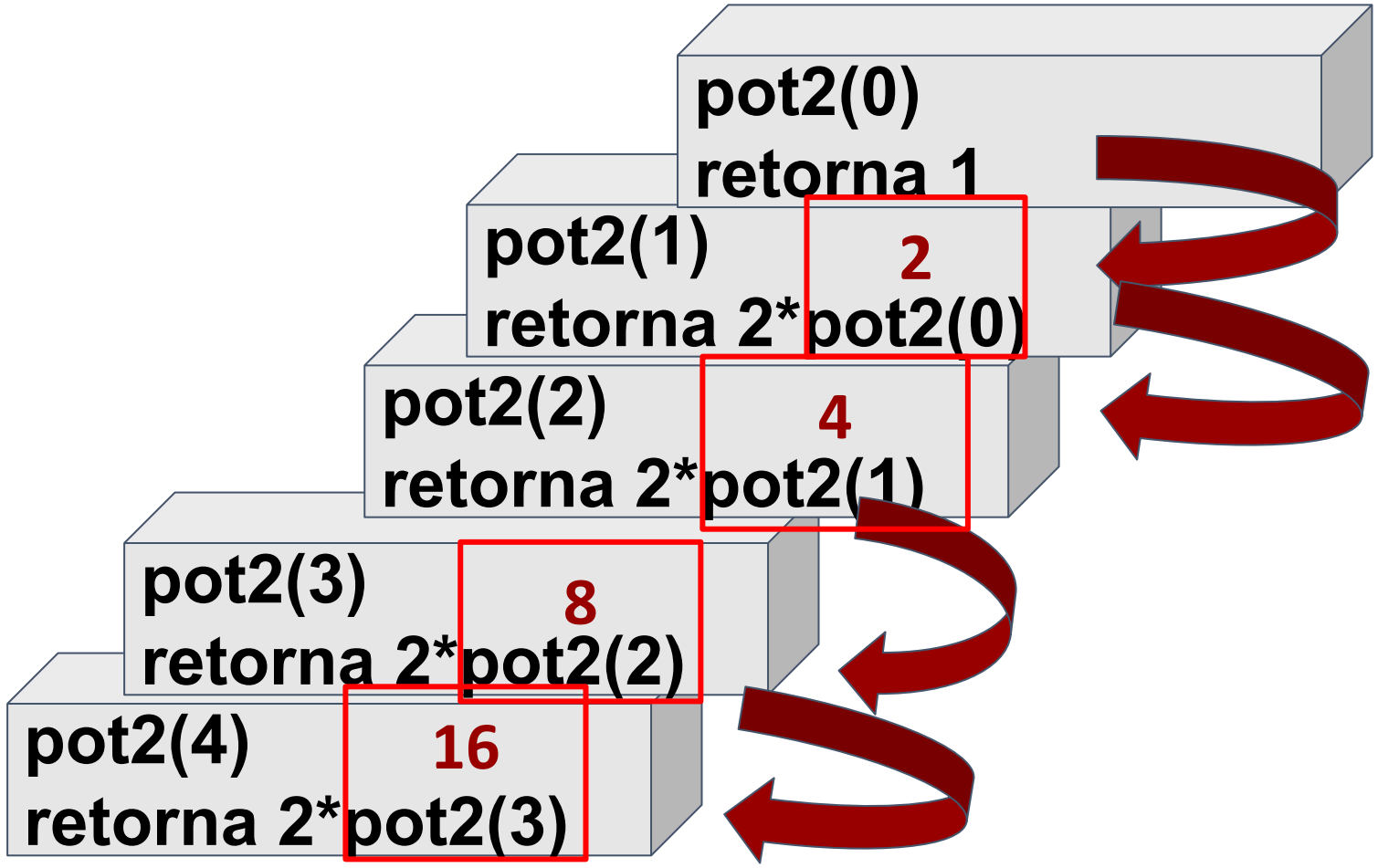
- Uma algoritmo é recursivo quando chama a si mesmo.
- Exemplo:

```
def pot2(n):
    if (n==0):
        return 1
    return (2*pot2(n-1))
```

Passo base

Passo recursivo

Algoritmo Recursivo



- Exemplo:

```
def pot2(n):
  if (n==0):
     return 1
  return (2*pot2(n-1))
```

Algoritmo Recursivo

Exemplo: Sequência de Fibonacci.

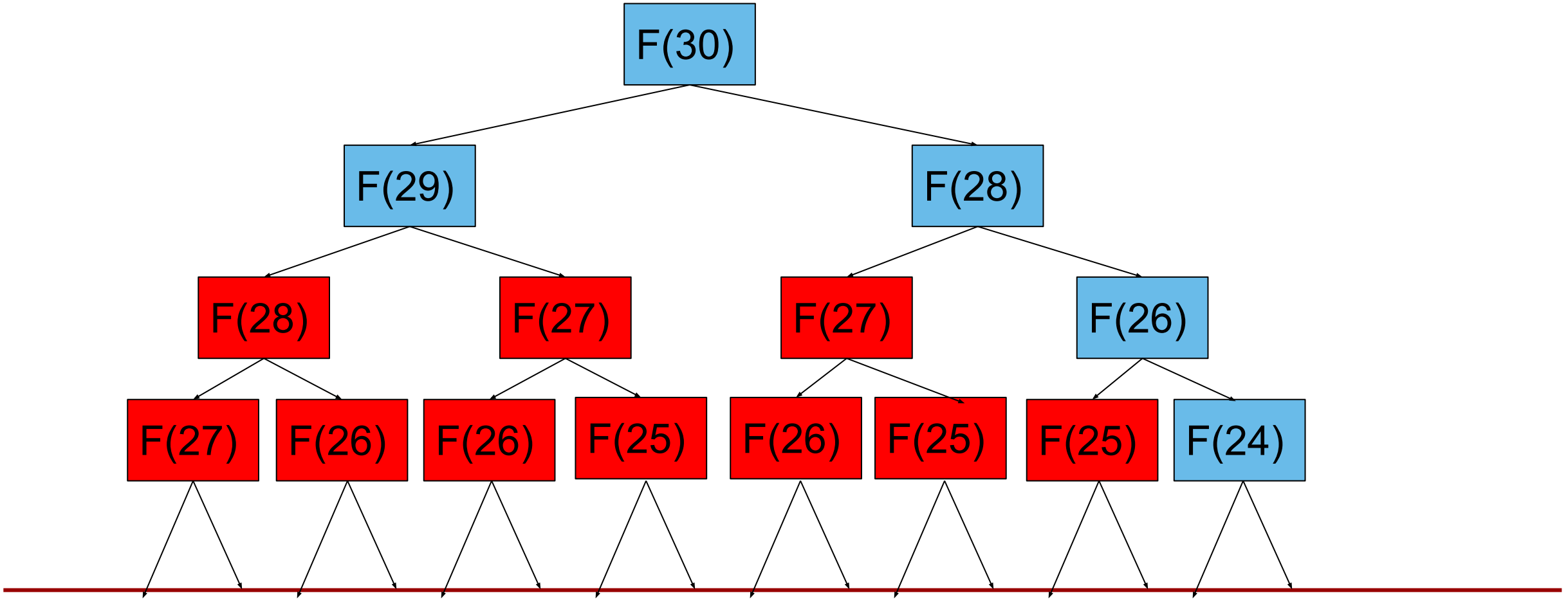
Passo base: f(0)=0, f(1)=1

Passo recursivo: f(n+1)=f(n)+f(n-1), n>1

```
def fib(int n):
    if(n<=1):
      return 1
    else:
      return (fib(n-1) + fib(n-2))
```

n	fib(n)	Número de chamadas da função
0	1	1
1	1	1
2	1	3
...	...	...
23	28657	92735
24	46368	150049

Algoritmo Recursivo



Algoritmo Recursivo

- Vantagens de Algoritmos Recursivos
  - Permitem modelar facilmente problemas mais complexos, onde características como resolução de subproblemas são tratadas de forma mais simples..
  - A recursividade descreve diretamente definições matemáticas, levando a implementação de códigos mais compreensíveis e concisos (redução no número de linhas).

Algoritmo Recursivo

- Desvantagens da Recursão
  - Gera sobrecarga (overhead) com as chamadas de função, levando a um gasto de tempo de processamento e espaço de memória.
  - Uma cópia da função (variáveis da função) é criada, consumindo memória.
  - Algoritmos iterativos tendem a ser mais rápidos por não fazerem repetidas chamadas às funções.

```
def pot2(n):
    if (n == 0):
        return 1
    return 2*pot2(n-1)
```

Algoritmo Recursivo

Vamos utilizar relações de recorrência para definir o custo de tempo dos algoritmos recursivos

1op: comparação -> 1
1op: retorno -> 1
1op: comparação -> 1
2op: retorno + produto + chamada função + subtração -> 5

Total(1) = 2 n=1
Total(n)= 5+T(n-1) n>1